

**REPUBLIC OF CAMEROON
PEACE-WORK-FATHERLAND
UNIVERSITY OF BUEA
BUEA, SOUTH-WEST REGION
P.O BOX 63.**



**RÉPUBLIQUE DU CAMEROUN
PAIX-TRAVAIL-PATRIE
UNIVERSITÉ DE BUEA
BUEA, RÉGION DU SUD-OUEST
B.P. 63.**

**FACULTY OF ENGINEERING AND TECHNOLOGY
DEPARTMENT OF COMPUTER ENGINEERING, SE
CEF440: INTERNET PROGRAMMING**

**DESIGN AND IMPLEMENTATION OF CHECKAR SMART
CAR DIAGNOSTICS MOBILE APPLICATION**

**Report by: GROUP 11
Department: Computer Engineering
Task 7: Final Project Presentation**

**Course Facilitator:
Dr.NKEMENI
VALERY**

**Date Issued:
3TH JULY 2026**

Presented by,

Name	Matricule
FUNWI KELSEA NDOHNWI	FE23A066
NGADANG ASSONFACK P. ESTELLE	FE23A109
ATEH SWIRRI FOFANG	FE23A018
TABOT GHISLAIN OROCK	FE23A146

ACKNOWLEDGEMENT

We wish to express our sincere gratitude to the Almighty for granting us the strength and wisdom to complete this project. We are deeply grateful to our supervisor, Dr. Nkemeni Valery, for his guidance, patience, and constructive feedback throughout the successive stages of this work, from requirements gathering through to system and database design.

ABSTRACT

Modern vehicles have grown increasingly complex, making early fault detection difficult for the average car owner and often forcing costly, avoidable visits to a mechanic simply to interpret a dashboard warning light or an unusual engine sound. This report presents CheckAR, an AI-powered mobile application designed to help car owners and mechanics in Cameroon diagnose common vehicle problems using dashboard-light image recognition and engine-sound classification, built on Flutter, Django, PostgreSQL, and TensorFlow. The project followed a structured, six-task development pathway: Task 1 reviewed mobile application development processes, platforms, programming languages, frameworks and architectural patterns to select an appropriate technical approach; Task 2 identified project stakeholders and used interviews, questionnaires, brainstorming, and reverse engineering of existing OBD-II applications to gather and clean requirements data while identifying sources of user reluctance; Task 3 analyzed and prioritized the gathered requirements using the MoSCoW method and consolidated them into a formal Software Requirements Specification (SRS); Task 4 modeled the system's behaviour and scope through a Context Diagram, a Data Flow Diagram, and a Use Case Diagram; Task 5 produced a complete UI/UX design system including brand identity, colour palette, typography, and sixteen screen mockups built around a minimal, easy-to-implement visual language; and Task 6 designed the underlying data architecture, comprising a twelve-entity Entity-Relationship model, a PostgreSQL database schema, and a three-layer Flutter → Django/REST → PostgreSQL backend strategy secured with JWT authentication. Taken together, these six tasks produced a complete, evidence-based design specification for CheckAR, validated against stakeholder needs and ready to guide the coding, testing, and deployment phases that constitute the project's remaining work. Keywords: mobile application development, car diagnostics, requirements engineering, software requirements specification, UI/UX design, entity-relationship modelling, Flutter, Django, PostgreSQL, machine learning.

TABLE OF CONTENTS

Contents

DESIGN AND IMPLEMENTATION OF CHECKAR SMART CAR DIAGNOSTICS MOBILE APPLICATION	1
Presented by,	2
ACKNOWLEDGEMENT	3
ABSTRACT.....	4
TABLE OF CONTENTS.....	5
LIST OF TABLES.....	8
LIST OF FIGURES	9
LIST OF ABBREVIATIONS.....	10
CHAPTER 1: GENERAL INTRODUCTION	11
1.1 Background and Context of the Study	11
1.2 Problem Statement.....	11
1.3 Objectives of the Study.....	12
1.3.1 General Objective	12
1.3.2 Specific Objectives	12
1.4 Proposed Methodology	13
1.5 Significance of the Study	13
1.6 Scope of the Study	14
1.7 Delimitation of the Study.....	14
1.8 Definition of Keywords and Terms	14
1.9 Organization of the Report.....	14
CHAPTER 2: LITERATURE REVIEW	16
2.1 Introduction.....	16
2.2 General Concepts in Mobile Application Development.....	16
2.2.1 The Mobile Application Development Process	16
2.2.2 Categories of Mobile Applications	16
2.2.3 Mobile Application Architectural Patterns	17
2.2.4 Frameworks, Languages, and Cost Considerations	17
2.2.5 The Requirement Engineering Process.....	18
2.3 Related Works: Existing Car Diagnostic Applications.....	18
2.3.1 FIXD OBD2 Scanner App	18
2.3.2 Car Scanner ELM OBD2	18
2.3.3 Identified Gaps in Existing Systems.....	18
2.4 Partial Conclusion.....	19

CHAPTER 3: ANALYSIS AND DESIGN	20
3.1 Introduction.....	20
3.2 Stakeholder Identification	20
3.3 Requirement Gathering Techniques	20
3.3.1 Interviews.....	20
3.3.2 Questionnaires and Surveys	20
3.3.3 Brainstorming	21
3.3.4 Reverse Engineering	21
3.4 Data Gathering, Cleaning, and User Reluctance	21
3.5 Requirements Analysis	22
3.5.1 Prioritization (MoSCoW).....	22
3.5.2 Sample of Finalized Functional Requirements	22
3.5.3 Non-Functional Requirements	23
3.6 Software Requirements Specification (SRS)	23
3.6.1 Scope and Product Perspective	23
3.6.2 User Classes	24
3.6.3 System Features	24
3.6.4 Validation and Testing Strategy.....	24
3.7 Requirements Validation with Stakeholders.....	25
3.8 System Architecture.....	25
3.8.1 Context Diagram.....	25
3.8.2 Data Flow Diagram.....	26
3.8.3 Use Case Diagram.....	27
3.9 UI/UX Design	28
3.9.1 App Identity and Visual Design System.....	28
3.9.2 Screen Designs.....	29
3.10 Database and Backend Design.....	31
3.10.1 Entity-Relationship Model.....	31
3.10.2 Database Implementation.....	32
3.11 Partial Conclusion.....	33
CHAPTER 4: IMPLEMENTATION (REALIZATION) AND RESULTS	34
4.1 Introduction.....	34
4.2 Tools and Materials Used	34
4.3 Description of the Implementation (Design) Process	34
4.4 Presentation and Interpretation of Results	35
4.5 Evaluation of the Solution	36
4.6 Partial Conclusion.....	36

CHAPTER 5: CONCLUSION AND FURTHER WORKS	38
5.1 Summary of Findings.....	38
5.2 Contribution to Engineering and Technology.....	38
5.3 Recommendations.....	38
5.4 Difficulties Encountered	39
5.5 Further Works	39
REFERENCES	41

LIST OF TABLES

Table 3.1: Summary of Identified Stakeholders

Table 3.2: Sample of Finalized Functional Requirements (SRS Section 5.3.1)

Table 3.3: Non-Functional Requirements Summary

Table 3.4: MoSCoW Prioritization Summary

Table 4.1: CheckAR Backend Technology Stack

LIST OF FIGURES

Figure 3.1: Context Diagram of CheckAR

Figure 3.2: Data Flow Diagram (Level 0/1) of CheckAR

Figure 3.3: Use Case Diagram of CheckAR

Figure 3.4: CheckAR Application Logo

Figure 3.5: CheckAR Colour Palette

Figure 3.6: Login Screen Mockup

Figure 3.7: Home Screen Mockup

Figure 3.8: Scan Dashboard Screen Mockup

Figure 3.9: Record Engine Sound Screen Mockup

Figure 3.10: Sound Diagnosis Result Screen Mockup

Figure 3.11: Help & About Screen Mockup

Figure 3.12: CheckAR Entity-Relationship Diagram (12 Entities)

Figure 4.1: Three-Layer System Architecture (Flutter → Django/DRF → PostgreSQL)

LIST OF ABBREVIATIONS

Abbreviation	Meaning
API	Application Programming Interface
DFD	Data Flow Diagram
DRF	Django REST Framework
ER	Entity-Relationship
FR	Functional Requirement
JWT	JSON Web Token
ML	Machine Learning
MoSCoW	Must-have, Should-have, Could-have, Won't-have (prioritization method)
MVC	Model-View-Controller
MVP	Model-View-Presenter
MVVM	Model-View-ViewModel
NFR	Non-Functional Requirement
OBD-II	On-Board Diagnostics, version II
ORM	Object Relational Mapper
PWA	Progressive Web Application
SRS	Software Requirements Specification
UI/UX	User Interface / User Experience

CHAPTER 1: GENERAL INTRODUCTION

1.1 Background and Context of the Study

The increasing mechanical and electronic complexity of modern vehicles has made basic fault diagnosis progressively harder for the average driver. A vehicle today integrates dozens of interdependent electronic control units, sensors, and software subsystems, and even a minor malfunction can trigger a dashboard warning light or an unfamiliar engine sound whose meaning is not obvious to a non-specialist. In many developing regions, including Cameroon, car owners typically respond to such signals by visiting a mechanic even for issues that could be self-diagnosed, resulting in avoidable costs, lost time, and in some cases exploitation by unscrupulous repairers.

CheckAR was conceived as a response to this gap: an AI-powered mobile application that allows a car owner to photograph a dashboard warning light or record an engine sound and receive an instant, plain-language diagnosis together with a recommended course of action. The project was undertaken by Group 11 as part of the CEF440 (Internet Programming and Mobile Programming) course under the supervision of Dr. Nkemeni Valery, and was developed progressively across six tasks, from initial research through to database and backend design, as consolidated in this report.

1.2 Problem Statement

Car owners, particularly in low- and middle-income contexts such as Cameroon, frequently lack the technical knowledge to interpret dashboard warning lights or unusual engine sounds. This forces reliance on mechanics for issues that may be minor, exposes owners to unnecessary expense, and can delay attention to genuinely urgent faults when access to a mechanic is not immediate. Existing OBD-II scanner applications on the market (such as FIXD and Car Scanner ELM OBD2) generally require a physical Bluetooth or Wi-Fi diagnostic dongle

plugged into the vehicle's OBD-II port, a barrier for owners of older vehicles without such ports, or those unable to afford the additional hardware. There is therefore a need for a diagnostic tool that works directly from a smartphone's camera and microphone, without dependence on external hardware, and that communicates results in accessible, non-technical language.

1.3 Objectives of the Study

1.3.1 General Objective

The general objective of this project is to design and specify CheckAR, an AI-powered mobile application that enables car owners and mechanics to diagnose common vehicle faults from dashboard warning-light images and engine-sound recordings, without requiring OBD-II hardware.

1.3.2 Specific Objectives

1. To review mobile application development approaches, platforms, and architectural patterns and select an appropriate technical strategy for the project (Task 1).
2. To identify the project's stakeholders and gather authentic requirements data through interviews, questionnaires, brainstorming, and reverse engineering of comparable existing applications (Task 2).
3. To analyze, clean, prioritize, and formally specify the gathered requirements in a Software Requirements Specification (Task 3).
4. To model the system's scope and behaviour through context, data-flow, and use-case diagrams (Task 4).
5. To design a coherent, minimal, and easy-to-implement user interface and user experience for the application (Task 5).

6. To design the underlying data model, database schema, and backend architecture that will support the application (Task 6).
7. To consolidate the outcomes of Tasks 1-6 into a single, coherent final project report (Task 7, this document).

1.4 Proposed Methodology

The project followed an iterative, requirements-driven methodology structured around the six preceding tasks. Primary data was collected directly from car owners and mechanics through bilingual (English/French) interviews and Google Forms surveys, supplemented by a brainstorming session among group members and a reverse-engineering review of two existing OBD-II diagnostic applications. This data informed a formal requirements analysis using MoSCoW prioritization, from which a full Software Requirements Specification was produced. The specification then guided system architecture modelling (context, data-flow, and use-case diagrams), UI/UX design (brand identity, design system, and sixteen screen mockups), and database/backend design (a twelve-entity Entity-Relationship model and a three-layer Flutter-Django-PostgreSQL architecture). Each stage was validated against the objectives and requirements established in the prior stage before progressing to the next.

1.5 Significance of the Study

This project is significant both academically and practically. Academically, it demonstrates a complete, evidence-based software engineering process from stakeholder analysis through to database design applied to a real-world problem. Practically, a successful CheckAR implementation would reduce the cost and time burden of vehicle fault diagnosis for car owners, particularly in contexts where OBD-II scanner hardware is inaccessible or unaffordable, while also creating a channel through which mechanics can be discovered and engaged for confirmed repairs.

1.6 Scope of the Study

The scope of this report covers the analysis and design phases of the CheckAR project: mobile application research (Task 1), stakeholder and requirements analysis (Tasks 2-3), system architecture modelling (Task 4), UI/UX design (Task 5), and database/backend design (Task 6). It does not cover the coding, testing, and deployment of a working software build, which are identified as further work in Chapter 5.

1.7 Delimitation of the Study

The study is delimited to the design of a mobile diagnostic application targeting two categories of vehicle faults: dashboard warning-light indicators and engine-sound anomalies. Diagnostic methods requiring physical connection to a vehicle's OBD-II port were deliberately excluded from the design, as one of the project's founding goals is to remove the dependency on such hardware.

1.8 Definition of Keywords and Terms

CheckAR: the AI-powered smart car diagnostics mobile application designed in this project.

Dashboard warning light: an illuminated icon on a vehicle's instrument cluster indicating a system status or fault.

OBD-II: On-Board Diagnostics version II, the standard vehicle diagnostic port and protocol.

Diagnostic session: a single instance of a user submitting an image or audio recording for analysis.

MoSCoW: a requirements prioritization technique classifying requirements as Must-have, Should-have, Could-have, or Won't-have.

1.9 Organization of the Report

This report is organized into five chapters. Chapter 1 (this chapter) introduces the project's background, problem, objectives, and scope. Chapter 2 reviews relevant literature on mobile application development and related diagnostic tools (drawn from Task 1 and the reverse-engineering component of Task 2). Chapter 3 presents the full analysis and design work of the project, covering stakeholder identification, requirements gathering and analysis, the Software Requirements Specification, system architecture, UI/UX design, and database design (Tasks 2 through 6). Chapter 4 presents the implementation strategy and the design outputs produced to date, evaluated against the project's objectives. Chapter 5 concludes the report with a summary of findings, contributions, difficulties encountered, and recommendations for further work.

CHAPTER 2: LITERATURE REVIEW

2.1 Introduction

This chapter reviews the general concepts of mobile application development that informed the technical direction of CheckAR (drawn from Task 1), followed by a review of existing, related diagnostic applications and the gaps they leave unaddressed (drawn from the reverse-engineering component of Task 2).

2.2 General Concepts in Mobile Application Development

2.2.1 The Mobile Application Development Process

Mobile application development follows a broadly recognized lifecycle: idea and planning, requirement gathering and analysis, UI/UX design, development (coding), testing, deployment, and ongoing maintenance and updates. CheckAR's own project structure across Tasks 1-6 mirrors this lifecycle, with Tasks 2-3 corresponding to requirement gathering and analysis, Task 4 to architectural design, Task 5 to UI/UX design, and Task 6 to the technical design that precedes development.

2.2.2 Categories of Mobile Applications

Mobile applications are commonly classified into native, hybrid, cross-platform, and progressive web applications (PWAs), each with distinct trade-offs in performance, platform compatibility, hardware accessibility, and development cost. Native applications, built with platform-specific languages such as Java or Kotlin for Android and Swift or Objective-C for iOS, offer the best performance and deepest hardware access but require separate codebases per platform. Hybrid and cross-platform frameworks, by contrast, allow a single codebase to target multiple platforms, trading a small amount of performance for significantly reduced development effort. PWAs run inside a browser and require no installation, but offer the weakest access to native device hardware such as the camera and microphone that CheckAR depends on.

Given that CheckAR's core functionality capturing dashboard images and recording engine audio depends on reliable access to the device camera and microphone, and given the team's goal of targeting Android with a single, maintainable codebase, a cross-platform framework was identified as the most suitable approach, leading to the eventual selection of Flutter (discussed further in Section 2.2.4).

2.2.3 Mobile Application Architectural Patterns

Three architectural patterns were reviewed for structuring the application's codebase: Model-View-Controller (MVC), Model-View-Presenter (MVP), and Model-View-ViewModel (MVVM). MVC is simple to learn and supports parallel development but can suffer from tight View-Controller coupling as an application grows. MVVM improves on this by promoting a clear separation between UI and business logic through data binding, which improves testability and maintainability at the cost of a steeper learning curve and additional boilerplate. Because CheckAR's UI will need to react to asynchronous, ML-driven diagnostic results, an MVVM-style separation of concerns was favoured in the project's design thinking, to keep the presentation layer decoupled from the diagnostic and networking logic.

2.2.4 Frameworks, Languages, and Cost Considerations

A comparative review of mobile programming languages and frameworks (Java, Kotlin, Swift, Objective-C, JavaScript, Dart) found Flutter, using the Dart language, to be well suited to CheckAR: it produces natively-compiled applications from a single codebase, offers strong support for custom, camera- and audio-driven interfaces, and has an active framework provider ecosystem. The review also considered mobile application cost estimation within the Cameroonian context, noting that payment gateway integration, mobile-OS maintenance overhead (“technical debt”), cloud infrastructure scalability amid currency volatility, and dependency on third-party APIs are all significant, recurring cost factors that a locally-deployed diagnostic application such as CheckAR must plan for.

2.2.5 The Requirement Engineering Process

The requirement engineering process — comprising elicitation, analysis and negotiation, specification, validation, and management — provided the methodological backbone for Tasks 2 and 3 of this project. Each of these five stages is reflected directly in the project's own process: elicitation through interviews and surveys, analysis and negotiation through issue identification and MoSCoW prioritization, specification through the formal SRS, and validation through structured stakeholder review, as detailed in Chapter 3.

2.3 Related Works: Existing Car Diagnostic Applications

As part of Task 2, two widely-used existing applications were reverse-engineered to identify their strengths, limitations, and the gaps CheckAR could address.

2.3.1 FIXD OBD2 Scanner App

FIXD pairs with a proprietary Bluetooth OBD-II dongle to read and clear diagnostic trouble codes, translating cryptic codes into plain-language explanations and repair-cost estimates. Its strengths lie in accuracy (since it reads directly from the vehicle's onboard computer) and ease of use once paired. Its principal limitation, from CheckAR's perspective, is its total dependence on a proprietary hardware accessory, which represents an upfront cost and a compatibility barrier for older vehicles.

2.3.2 Car Scanner ELM OBD2

Car Scanner ELM OBD2 offers a similar function using generic, more affordable ELM327 Bluetooth/Wi-Fi adapters, and additionally exposes live sensor data (RPM, speed, temperature) as customizable dashboards. It is more flexible and lower-cost than FIXD in terms of hardware, but still requires a physical adapter, a technical setup process, and a vehicle with a functioning OBD-II port — a requirement that excludes older vehicles and users unfamiliar with connecting external hardware.

2.3.3 Identified Gaps in Existing Systems

- Both reviewed applications require a physical OBD-II adapter, excluding vehicles without a working port and adding hardware cost for the user.
- Neither application diagnoses faults from dashboard warning-light images or engine sound alone, both of which car owners can capture with an ordinary smartphone.
- Neither offers an integrated pathway to a nearby mechanic or repair-tutorial content once a fault is identified.
- Setup and use of existing OBD-II scanner apps assumes a baseline of technical comfort that many car owners in the target market do not have.

These gaps directly shaped CheckAR's core value proposition: hardware-free diagnosis from images and sound, paired with plain-language explanations, mechanic discovery, and tutorial links.

2.4 Partial Conclusion

The literature and comparative application review confirmed that a cross-platform, Flutter-based mobile application, structured around an MVVM-style separation of concerns and built without dependence on OBD-II hardware, represents a sound and differentiated technical direction for CheckAR. These findings, together with the stakeholder and requirements data discussed in the next chapter, directly informed the system's functional scope and architecture.

CHAPTER 3: ANALYSIS AND DESIGN

3.1 Introduction

This chapter consolidates the analysis and design work carried out across Tasks 2 through 6 of the project. It covers stakeholder identification and requirements gathering, requirements analysis and the resulting Software Requirements Specification, requirements validation, system architecture modelling, UI/UX design, and database/backend design.

3.2 Stakeholder Identification

Seven categories of stakeholders were identified for the CheckAR project, each with distinct interests and influence on the system's requirements.

Stakeholder	Role / Interest
Car Owners / Drivers	Primary end users seeking accessible, low-cost fault diagnosis.
Mechanics & Automotive Technicians	Secondary users who validate diagnoses and receive referrals.
System Developers	Group 11 members responsible for designing and building the system.
Vehicle Maintenance Experts / Content Creators	Provide tutorial and repair-guidance content.
Automobile Companies & Diagnostic Data Providers	Sources of reference fault-code and vehicle data.
Internet Service Providers / Cloud Service Providers	Provide the connectivity and hosting infrastructure.

Table 3.1: Summary of Identified Stakeholders

3.3 Requirement Gathering Techniques

3.3.1 Interviews

Structured interviews were conducted with car owners and mechanics to understand real-world diagnostic pain points, existing habits when a warning light appears, and attitudes toward a smartphone-based diagnostic tool. The interviews confirmed that most car owners lack the technical vocabulary to describe a fault precisely, and that mechanics are generally open to a tool that pre-screens likely faults before a customer visit.

3.3.2 Questionnaires and Surveys

Bilingual (English/French) Google Forms surveys were distributed separately to car owners and to car repairers, covering knowledge of common car faults, user behaviour and habits,

experience interpreting engine sounds, willingness to adopt a mobile diagnostic solution, desired features, and technical constraints such as smartphone camera/microphone quality and internet access. The survey responses formed the primary evidence base used later to justify and prioritize functional requirements (see Section 3.4).

3.3.3 Brainstorming

A structured brainstorming session among Group 11 members was held to expand on survey and interview findings, generating candidate features such as user guidance and recommendations, offline/online functionality, and diagnostic history and records — several of which were later formalized as functional requirements in the SRS.

3.3.4 Reverse Engineering

As discussed in Chapter 2, two existing OBD-II diagnostic applications (FIXD and Car Scanner ELM OBD2) were reverse-engineered to extract their feature sets and identify functionality gaps that CheckAR could fill, most notably hardware-free diagnosis from images and audio.

3.4 Data Gathering, Cleaning, and User Reluctance

Raw responses collected from interviews and bilingual surveys were consolidated and cleaned before analysis. This involved merging English and French responses into a single dataset, resolving multiple-selection answers, removing irrelevant or ambiguous open-text entries, normalizing inconsistent response formatting, and verifying the completeness of each response before it was accepted into the analysis dataset.

The data gathering process also surfaced patterns of user reluctance that were factored into the system design, summarized in Table 3.1a below.

Reluctance Pattern	Description
Accuracy-Trust Paradox	Users want AI diagnosis but distrust it unless it matches a mechanic's judgment.
Technical Intimidation	Some users doubt their ability to correctly photograph or record a fault.

Safety & Liability Concerns	Users worry about acting on a diagnosis that turns out to be wrong.
Environmental / Infrastructural Barriers	Poor lighting, background noise, and unreliable internet affect usability.

These findings directly motivated several SRS requirements, including offline diagnostic capability (FR13), confidence-scored results, and plain-language explanations rather than raw technical codes.

3.5 Requirements Analysis

Raw functional and non-functional requirements extracted from user responses were reviewed, deduplicated, and rewritten as clear, testable requirement statements. Ambiguous or overlapping items identified during this review were resolved before the requirements were formally classified and specified.

3.5.1 Prioritization (MoSCoW)

All finalized requirements were prioritized using the MoSCoW technique — Must-have, Should-have, Could-have, Won't-have — based on their importance to core diagnostic functionality and technical feasibility within the project timeline. Must-have requirements include user authentication, dashboard-light image capture and interpretation, engine-sound recording and classification, and offline diagnostic capability. Should-have requirements include tutorial links, diagnostic history, and maintenance reminders. Could-have requirements include report sharing and mechanic professional notes.

3.5.2 Sample of Finalized Functional Requirements

ID	Requirement	Priority
FR1	The app shall provide user registration functionality with email verification.	Must-Have
FR2	The app shall support role-based authentication (car owner, mechanic, administrator).	Must-Have
FR5	The app shall allow users to capture dashboard images and detect warning-light icons.	Must-Have
FR7	The app shall record engine sound clips up to 10 seconds using the device microphone.	Must-Have
FR8	The app shall classify engine sounds into predefined fault categories.	Must-Have

FR9	The app shall provide plain-language explanations and repair suggestions for detected faults.	Must-Have
FR10	The app shall include links to relevant video tutorials for each diagnosed issue.	Should-Have
FR13	The app shall operate offline for image and audio diagnostics using on-device models.	Must-Have
FR17	The app shall estimate maintenance urgency on a three-tier scale (Immediate, Soon, Monitor).	Must-Have
FR21	The app shall provide administrators with full user-management capabilities.	Must-Have

Table 3.2: Sample of Finalized Functional Requirements (SRS Section 5.3.1)

3.5.3 Non-Functional Requirements

Category	Summary
Performance	Diagnostic results must be returned within a short, defined response time for both online and on-device processing.
Reliability	The system must consistently produce diagnoses without crashing or losing user data.
Usability	The interface must be usable by non-technical car owners, in line with the minimal design philosophy of Task 5.
Security	User data and authentication must follow secure practices, including JWT-based sessions.
Maintainability & Portability	The codebase must be structured for straightforward maintenance and future platform portability.

Table 3.3: Non-Functional Requirements Summary

3.6 Software Requirements Specification (SRS)

A formal SRS (Version 1.0, Draft) was produced following an IEEE 830-informed structure, covering purpose, scope, definitions, product perspective, product functions, user classes, assumptions and dependencies, specific functional and external interface requirements, system features, and validation/testing strategy.

3.6.1 Scope and Product Perspective

CheckAR is specified as a standalone, Android-first mobile application with an accompanying backend service, positioned as a companion tool to — not a replacement for — professional automotive mechanics. It is intended for use by car owners with little or no automotive

technical background, by mechanics who wish to pre-screen reported faults, and by administrators who manage user accounts and diagnostic model updates.

3.6.2 User Classes

- Primary: Car Owners (non-technical) the main users capturing images/audio and reading diagnoses.
- Secondary: Mechanics: users who may review diagnoses and add professional notes.
- Administrative Users: manage accounts, content, and machine-learning model versions.

3.6.3 System Features

- User Authentication System registration, login, password recovery, and session management.
- Dashboard Warning Light Scanner image: capture and icon detection/interpretation.
- Engine Sound Analyzer: audio capture and fault-category classification.
- Diagnosis and Reporting: plain-language results, urgency levels, and shareable reports.
- Maintenance Tracking: scheduling and reminders based on diagnoses or user-defined intervals.
- Online/Offline Functionality: on-device diagnosis when offline, with sync when connectivity resumes.

3.6.4 Validation and Testing Strategy

The SRS defines a testing strategy to be executed once implementation begins, spanning unit testing of individual diagnostic and authentication components, integration testing of the Flutter-to-backend communication, and user acceptance testing with car owners and mechanics drawn from the original interview and survey pool.

3.7 Requirements Validation with Stakeholders

The finalized requirements and SRS were reviewed against the original interview and survey data captured in Task 2 to confirm that no critical stakeholder need had been omitted and that no requirement contradicted user expectations. This review also surfaced project risks — such as the technical difficulty of achieving high accuracy in on-device sound classification, and the risk of low user trust in AI-generated diagnoses — together with corresponding mitigation strategies, including confidence-score displays and clear disclaimers recommending professional confirmation for high-urgency faults.

3.8 System Architecture

System architecture modelling (Task 4) translated the SRS into a set of structural and behavioural diagrams describing how CheckAR interacts with external entities and how data flows through the system.

3.8.1 Context Diagram

The Context Diagram presents CheckAR as a single process exchanging data with five external entities: the Administrator, the User (driver/mechanic), Google Maps (for mechanic-location services), YouTube (for tutorial content), and an Authentication Service. It establishes the system's boundary without detailing internal processing.

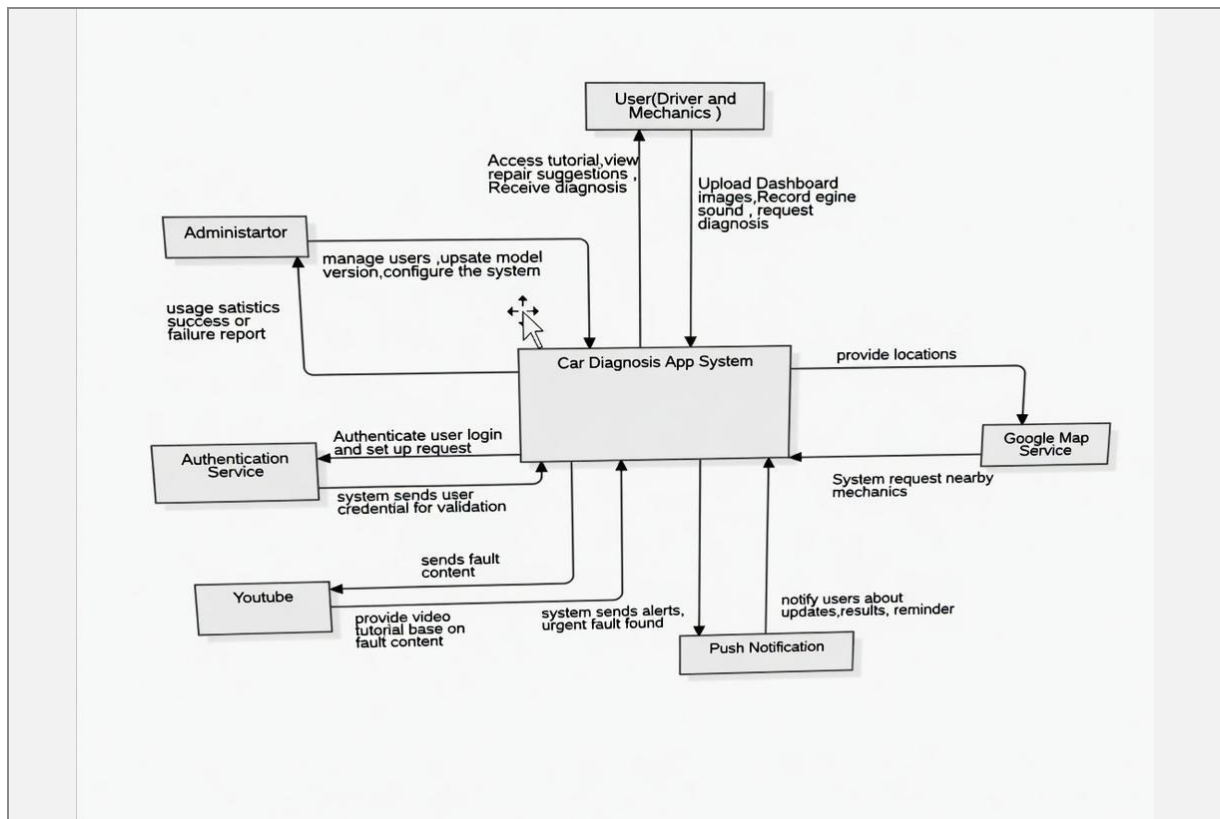


Figure 3.1: Context Diagram of CheckAR

3.8.2 Data Flow Diagram

The Data Flow Diagram decomposes the Context Diagram's single process into CheckAR's major internal processes authentication, image-based diagnosis, audio-based diagnosis, reporting, and maintenance tracking and shows how data moves between these processes, the system's data stores, and its external entities.

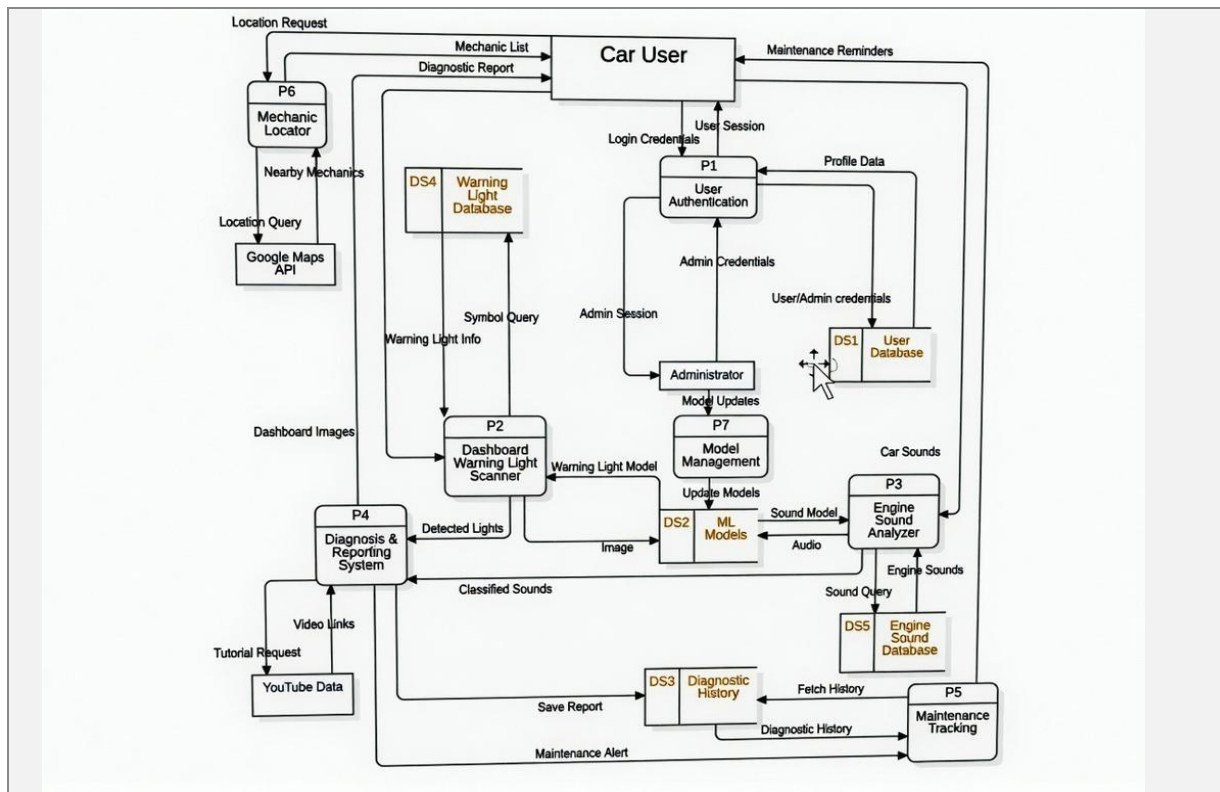


Figure 3.2: Data Flow Diagram of CheckAR

3.8.3 Use Case Diagram

The Use Case Diagram identifies two primary actors — the Car User and the System Administrator — and the use cases available to each. The Car User has access to registration, login, dashboard scanning, engine-sound recording, diagnosis viewing, maintenance tracking, and mechanic location services, while the Administrator is restricted to backend system management, user administration, and diagnostic model updates. Include relationships show, for example, that “View Diagnosis Report” includes “Scan Dashboard Warning Lights” and/or “Record Engine Sound”, and that “Share Diagnostic Reports” and “Find Nearby Mechanics” both include “View Diagnosis Report”.

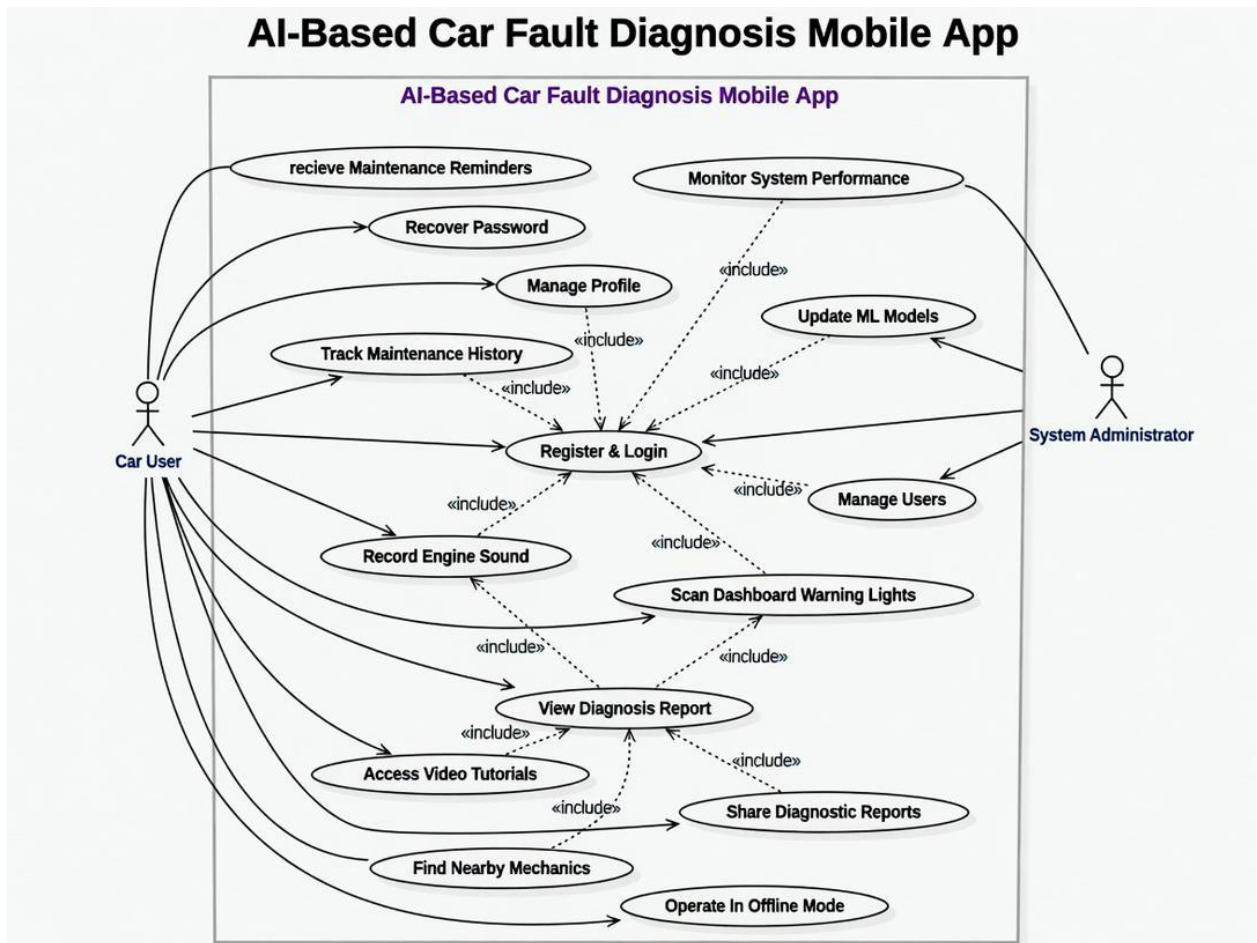


Figure 3.3: Use Case Diagram of CheckAR

These diagrams collectively validate that all functional requirements from the SRS are properly captured and assigned to an appropriate actor, and provide a technical blueprint that the development team can follow when coding begins.

3.9 UI/UX Design

Task 5 translated the SRS and architecture into a complete, minimal, and easy-to-implement visual design system for CheckAR, deliberately favouring clean, uncrowded screens that would be straightforward to build in Flutter.

3.9.1 App Identity and Visual Design System

The brand identity centres on a professional blue primary colour (#2563EB), chosen for its associations with trust and technical competence, supported by a small complementary palette of six core colours used consistently across the app. Typography is built around the Inter font

family with a clearly defined size hierarchy, and the layout follows a consistent grid and spacing scale, with a restricted set of functional icon categories to keep the interface visually coherent



Figure 3.4: CheckAR Application Logo

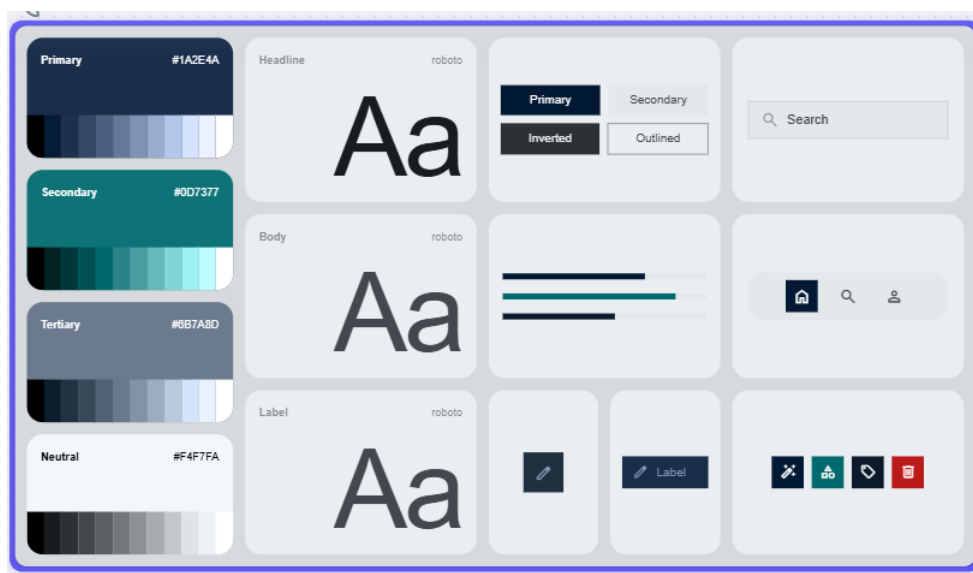


Figure 3.5: CheckAR Colour Palette

3.9.2 Screen Designs

Sixteen screens were mocked up in total, covering authentication, the home dashboard, both diagnostic capture flows, results, history, maintenance tracking, and settings. The five most central screens are illustrated below; the remaining mockups should be inserted in Appendix A.

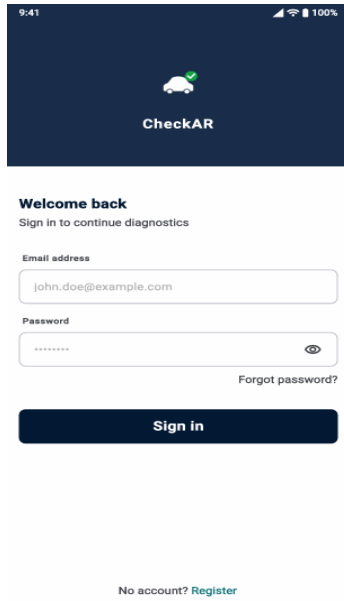


Figure 3.6: Login Screen Mockup

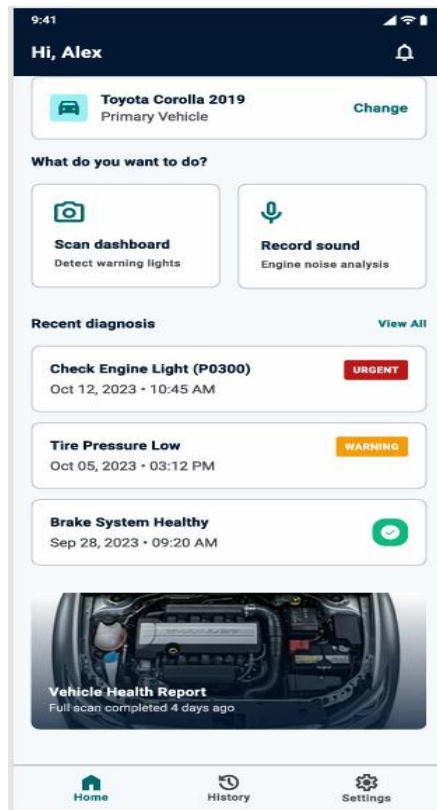


Figure 3.7: Home Screen Mockup

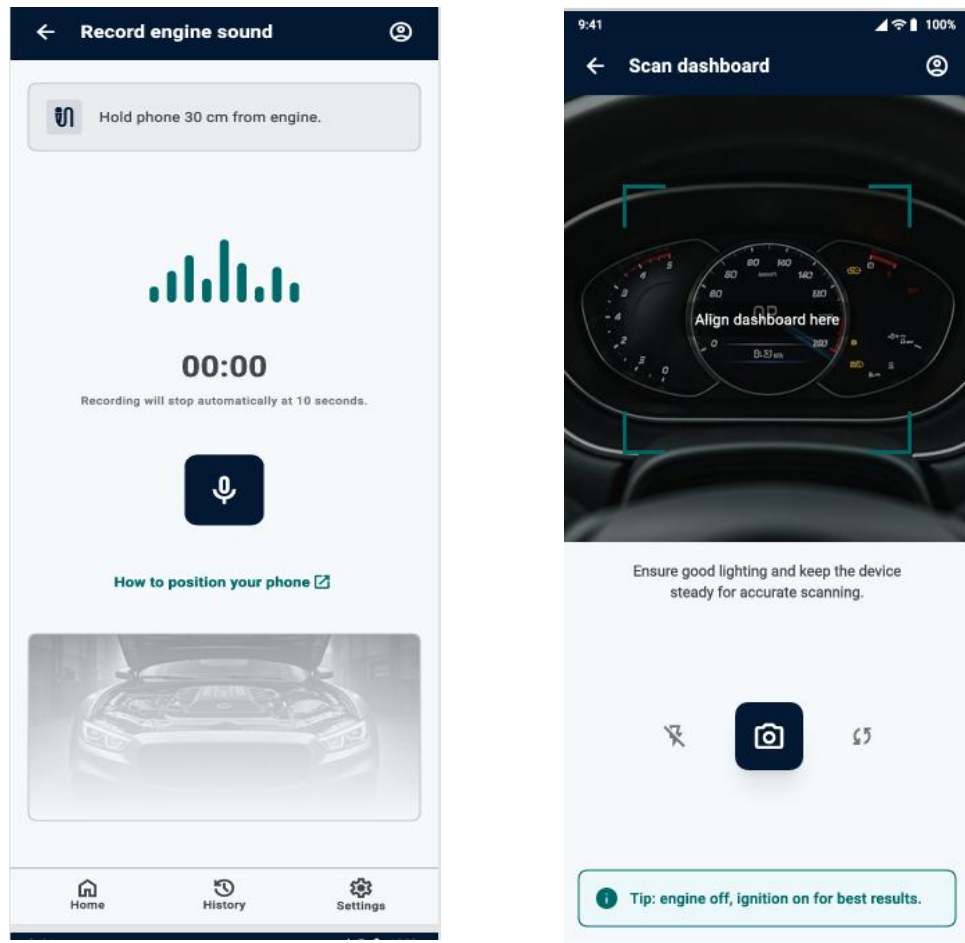


Figure 3.8: Scan Dashboard Screen Mockup

The design system was validated through a planned four-stage testing approach: prototype testing with wireframes, alpha testing with the internal team, beta testing with a limited external user group, and ongoing production testing through continuous user feedback.

3.10 Database and Backend Design

Task 6 defined the data architecture underpinning CheckAR, beginning with an analysis of the data elements needed to support every functional requirement in the SRS, followed by a conceptual design and a full Entity-Relationship model.

3.10.1 Entity-Relationship Model

The CheckAR data model comprises twelve core entities: Users, User_Profiles, Diagnostic_Records, Diagnostic_Results, Warning_Lights, Engine_Sounds, Maintenance_Records, Mechanics, Services, Tutorials, ML_Models, and Audit_Logs. The

Users entity is the central hub for account management; User_Profiles extends it in a one-to-one relationship for optional profile data; Diagnostic_Records and Diagnostic_Results together capture each diagnostic session and its AI-generated outcome, including confidence scores and urgency levels; Warning_Lights and Engine_Sounds serve as reference catalogues of known fault signatures; Maintenance_Records, Mechanics, and Services support the maintenance-tracking and mechanic-referral features; Tutorials links diagnoses to educational content; ML_Models tracks model versioning; and Audit_Logs records system activity for security and compliance.

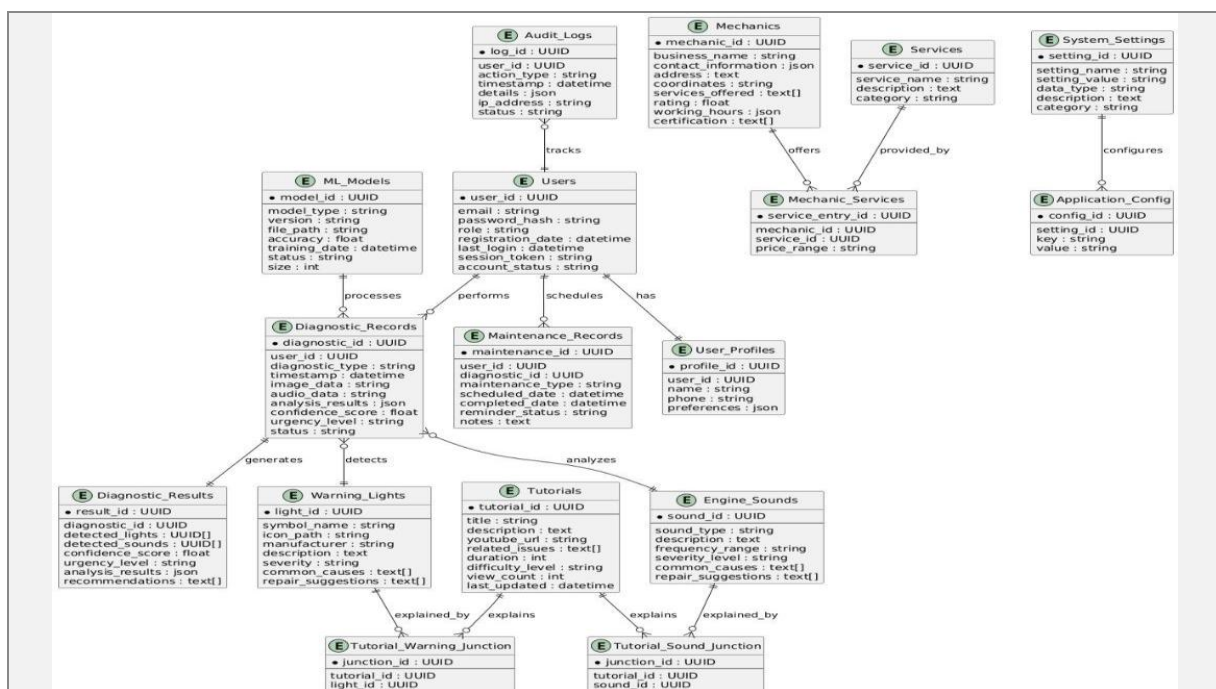


Figure 3.12: CheckAR Entity-Relationship Diagram (12 Entities)

3.10.2 Database Implementation

The physical database is implemented in PostgreSQL, chosen for its reliability and ability to scale with concurrent users and large datasets. Django's Object-Relational Mapper (ORM) is used to manage all database interactions, translating Python model classes into optimized SQL, which both reduces development complexity and mitigates the risk of SQL injection. Sensitive

configuration values, such as database credentials, are stored in environment variables rather than in source code.

3.11 Partial Conclusion

Chapter 3 has consolidated the full analysis and design pipeline of the CheckAR project: from stakeholder identification and requirements gathering, through formal specification and validation, to system architecture, UI/UX design, and database design. Each stage was directly traceable to evidence gathered from real car owners and mechanics, producing a coherent and well-justified specification ready to guide implementation, presented in the next chapter.

CHAPTER 4: IMPLEMENTATION (REALIZATION) AND RESULTS

4.1 Introduction

This chapter presents the implementation strategy defined for CheckAR and the concrete design outputs produced across Tasks 1-6, which together constitute the results of the project's analysis and design phase. As of this report, the project has completed the full design specification of the system; coding, integrated testing, and deployment of a working build are identified as further work in Chapter 5.

4.2 Tools and Materials Used

The following technology stack was selected during Tasks 1 and 6 to implement CheckAR.

Technology	Purpose
Flutter (Dart)	Cross-platform mobile front-end framework
Python	Backend programming language
Django	Web application framework (MVT pattern)
Django REST Framework (DRF)	REST API development
PostgreSQL	Relational database management system
TensorFlow	Machine learning model execution (image and audio classification)
JWT Authentication	Secure user authentication and session management
GitHub	Version control and team collaboration
Docker	Application containerization for consistent deployment

Table 4.1: CheckAR Backend Technology Stack

4.3 Description of the Implementation (Design) Process

The implementation strategy follows a three-layer architecture: the Flutter mobile client communicates with a Django/DRF backend over REST APIs secured by JWT authentication, and the backend in turn persists and retrieves data from a PostgreSQL database through Django's ORM. TensorFlow models for dashboard-light recognition and engine-sound

classification are intended to run both on-device (for offline diagnosis, per requirement FR13) and, where connectivity allows, on the backend for heavier processing and model updates.

Database connectivity is managed through Django's ORM configuration, with credentials such as database name, user, password, host, and port supplied via environment variables rather than hard-coded into the source, in line with the security requirements defined in the SRS. GitHub is used for version control and team collaboration across the four members of Group 11, and Docker is planned for containerizing the backend to ensure consistent behaviour across development and eventual deployment environments.

4.4 Presentation and Interpretation of Results

As coding has not yet begun, the “results” of the project to date are its design artifacts: the finalized SRS, the system architecture diagrams, the UI/UX design system and sixteen screen mockups, and the Entity-Relationship model and database schema, all presented in Chapter 3. Collectively, these artifacts demonstrate that:

- Every functional requirement in the SRS can be traced back to specific interview or survey evidence gathered in Task 2.
- Every use case in the Task 4 Use Case Diagram maps to at least one functional requirement, confirming complete requirements coverage.
- Every screen mocked up in Task 5 corresponds to a system feature defined in the SRS (authentication, dashboard scanning, sound recording, diagnosis results, and help/about).
- Every data element required by the functional requirements has a corresponding entity or attribute in the Task 6 Entity-Relationship model.

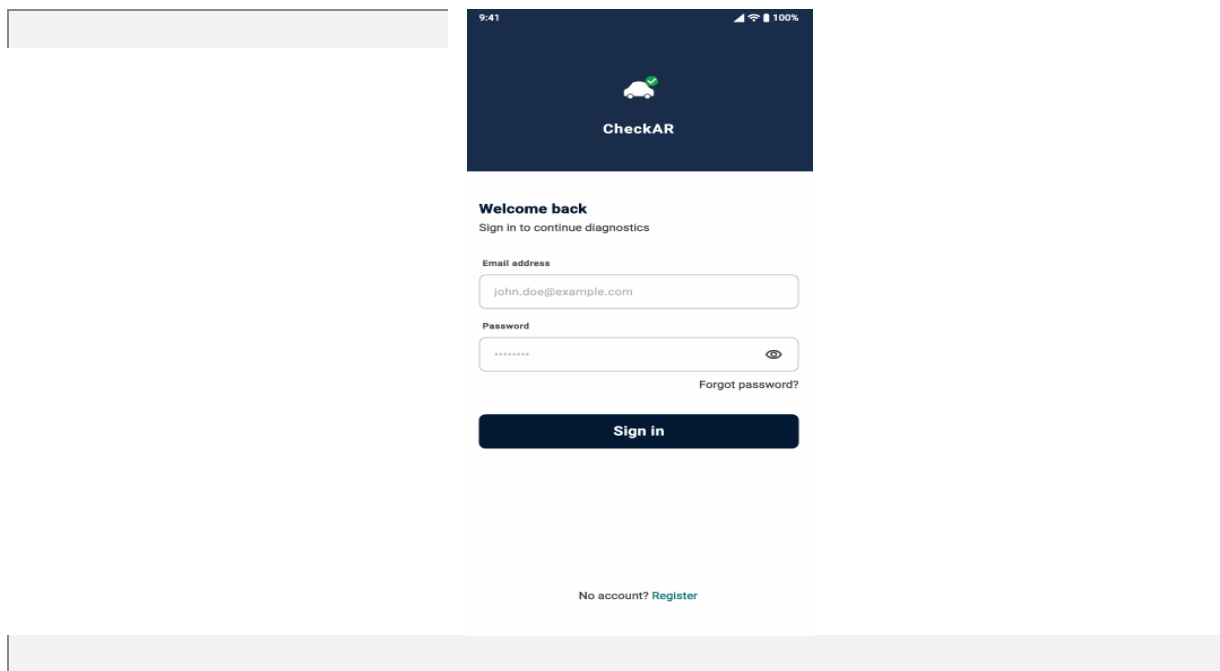


Figure 4.2 (example): Running Application Screenshot

4.5 Evaluation of the Solution

Measured against the objectives set out in Chapter 1, the project has, to date, fully satisfied the specific objectives relating to research, requirements gathering, requirements specification, architecture modelling, UI/UX design, and database design (Objectives 1 through 6). The general objective — a working application capable of diagnosing vehicle faults from images and sound — remains to be demonstrated through actual implementation and testing, since Tasks 1 through 6 constitute the analysis and design phase of the software development lifecycle rather than the build phase.

The planned validation approach, once a working build exists, follows the four-stage testing strategy defined in the SRS: prototype testing with wireframes, alpha testing internally within Group 11, beta testing with a limited group of the car owners and mechanics originally surveyed in Task 2, and continuous production testing through user feedback after deployment.

4.6 Partial Conclusion

This chapter has presented the technology stack and implementation strategy chosen for CheckAR, and evaluated the project's design outputs against its stated objectives. While the core software has not yet been built, the design specification produced across Tasks 1-6 provides a complete and internally consistent blueprint from which implementation can proceed directly, with minimal further analysis required.

CHAPTER 5: CONCLUSION AND FURTHER WORKS

5.1 Summary of Findings

This report has consolidated six tasks of work undertaken toward the design of CheckAR, an AI-powered smart car diagnostics mobile application. Task 1 established a sound technical foundation by reviewing mobile application development processes, categories, languages, frameworks, and architectural patterns. Task 2 identified seven categories of project stakeholders and gathered authentic requirements data through interviews, bilingual surveys, brainstorming, and reverse engineering of two existing OBD-II applications, while also surfacing patterns of user reluctance that shaped later design decisions. Task 3 analyzed, prioritized, and formally specified the gathered requirements in a complete SRS, validated against the original stakeholder data. Task 4 modelled the system's scope and behaviour through context, data-flow, and use-case diagrams. Task 5 produced a minimal, easy-to-implement UI/UX design system with sixteen screen mockups. Task 6 designed a twelve-entity data model and a three-layer Flutter-Django-PostgreSQL backend architecture.

5.2 Contribution to Engineering and Technology

This project contributes a complete, evidence-based analysis and design specification for a hardware-free vehicle diagnostic application — a category not directly addressed by existing OBD-II-dependent tools such as FIXD and Car Scanner ELM OBD2. By grounding every functional requirement in primary data collected from Cameroonian car owners and mechanics, the project demonstrates a rigorous, stakeholder-driven requirements engineering process, and produces a coherent architecture, interface design, and data model ready to guide direct implementation.

5.3 Recommendations

8. Proceed directly to implementation using the technology stack and architecture defined in Task 6, since the design specification is complete and internally validated.
9. Prioritize the Must-Have functional requirements identified in the MoSCoW analysis (authentication, image and sound diagnosis, offline capability) for the first working build.
10. Conduct early on-device testing of the TensorFlow models for dashboard-light recognition and sound classification, as model accuracy is the primary technical risk identified during requirements validation.
11. Re-engage the original interview and survey participants during beta testing to validate that the delivered application meets the expectations captured during Task 2.

5.4 Difficulties Encountered

- Reconciling bilingual (English/French) survey responses into a single, consistent dataset during data cleaning.
- Balancing a minimal, easy-to-implement UI design against the breadth of features requested by survey respondents.
- Anticipating the technical feasibility of on-device machine learning inference for offline diagnosis without yet having a working model to benchmark.
- Coordinating four group members across the six sequential tasks within the course timeline.

5.5 Further Works

12. Implementation of the Flutter mobile client and the Django/DRF backend as specified in Task 6.

13. Training and integration of the TensorFlow models for dashboard warning-light recognition and engine-sound classification.
14. Unit, integration, and user-acceptance testing following the strategy defined in the SRS and Task 5.
15. Deployment of the backend (e.g., via Docker) and publication of the Android application for beta testing with real users.
16. Post-launch monitoring and iterative refinement based on production user feedback, as outlined in the SRS validation strategy.

REFERENCES

CheckAR Requirements Analysis Report and Software Requirements Specification (SRS), Version 1.0, Group 11, University of Buea.

CheckAR Task 2: Stakeholder Analysis and Data Gathering Report, Group 11, University of Buea.

IEEE Std 830-1998. IEEE Recommended Practice for Software Requirements Specifications.

ISO/IEC 25010:2011. Systems and Software Quality Requirements and Evaluation (SQuaRE).

ISO 26262. Road Vehicles — Functional Safety Standard.

ISO 15005. Road Vehicles — Ergonomic Aspects of Transport Information and Control Systems — Dialogue Management Principles and Compliance Procedures.

Web Content Accessibility Guidelines (WCAG) 2.1, World Wide Web Consortium (W3C).

Flutter Documentation ---UI Development Best Practices, Google.

Material Design 3 — Google Design System Guidelines, Google.

Human Interface Guidelines, Apple Inc.

Nielsen Norman Group — Mobile UX Design Principles.

Django Software Foundation. Django Documentation.

PostgreSQL Global Development Group. PostgreSQL Documentation.

TensorFlow Documentation, Google.

FIXD Automotive Inc. FIXD OBD2 Scanner App (product documentation).

Car Scanner ELM OBD2, OBD Solutions (product documentation).

